PROJECT PROPOSAL

Explainable AI for Diabetes Risk Prediction

A Mobile-First, SHAP and LIME-Based Framework for Interpretable Diabetes Risk Screening

Sr. No.	Name	Details
1	Project Title	Explainable AI for Diabetes Risk Prediction
2	**Faculty Mentor**	
	Name & Dept.	Dr. Soni Sweta, Computer Engineering
	College Name	MPSTME, NMIMS University, Mumbai
3	**Student details**	
a	Team Leader (1) Name	Gaurav Suryakant Phadale
	Degree/Dept/Year	B.Tech, Computer Engineering, 4th Year (2026–27)
b	Member 2 Name	Eshaan Sarkhawas
	Degree/Dept/Year	B.Tech, Computer Engineering, 4th Year (2026–27)

Table of Contents

1. Executive Summary

Diabetes is one of the largest and fastest-growing chronic disease burdens in India. Machine learning models can already predict diabetes risk with strong accuracy, but almost all of them operate as black boxes: they return a score without explaining what is driving it. This proposal builds a framework that does both: it predicts diabetes risk using three benchmark classifiers (Logistic Regression, Random Forest and XGBoost), and then explains every individual prediction using SHAP and LIME, at both the population level and the single-user level.

The project is scoped around a single, clearly defined deliverable: a mobile application that a patient or a clinician can use to obtain a diabetes risk score together with a plain-language explanation of what is driving that score. Every methodological choice in this proposal, including the dataset, the models, the split ratio, the explainability layer and the technology stack, is made with that mobile deliverable as the design target, not as an afterthought bolted on at the end.

Core Deliverables

- A mobile application (Android/iOS via Flutter) that returns a risk score, a risk percentage, and a SHAP + LIME based explanation in plain language.
- Benchmarked models (Logistic Regression, Random Forest, XGBoost) evaluated with an identical, leakage-free protocol.
- A SHAP and LIME explainability report documenting global and local feature attributions.
- A technical research write-up benchmarked against 2025 diabetes-XAI literature.
- A deployment-ready package (FastAPI backend + trained model artifacts + mobile client).

Present Relevance

India accounted for an estimated 89.8 million adults living with diabetes in 2024, the second-highest national burden in the world after China, with close to 43 percent of cases undiagnosed (International Diabetes Federation, IDF Diabetes Atlas, 11th Edition, 2025). Explainability is also shifting from a research nicety to a compliance expectation internationally: the EU AI Act (Regulation (EU) 2024/1689) classifies healthcare AI as high-risk and requires meaningful information about a model's decision logic from August 2026 onward. This project is built around the idea that a risk score without a reason is of limited clinical use, whether the audience is a patient deciding whether to see a doctor, or a doctor deciding how to counsel a patient.

2. Background and Motivation

2.1 Diabetes: The Scale of the Problem

Diabetes is a leading chronic disease burden in India and globally, and already has a rich base of 2024-2026 explainable ML literature to build on, benchmark against, and differentiate from.

- India had an estimated 89.8 million adults (aged 20-79) living with diabetes in 2024, ranking second globally after China (IDF Diabetes Atlas, 11th Edition, 2025).
- India's age-standardised diabetes prevalence stood at 10.5 percent in 2024 and is projected to rise to 12.8 percent by 2050, with the total diabetic population projected to reach 156.7 million by 2050, a 75 percent increase (IDF Diabetes Atlas, 11th Edition, 2025).
- Approximately 43 percent of diabetes cases in India remain undiagnosed, meaning a large share of at-risk individuals are never flagged until complications have already begun (IDF Diabetes Atlas, 11th Edition, 2025).
- A 2026 Frontiers in Medicine review notes that most existing ML-based risk models are developed and validated on European and North American cohorts, creating a critical lack of representativeness for Asian populations, a direct, recent, published validation of the India-specific data gap this proposal treats as an explicit limitation (Section 21), not an afterthought.

2.2 The Explainability Gap in Clinical ML

Across the papers reviewed for this proposal (Section 5), a consistent pattern emerges: predictive accuracy has largely been solved, with several studies reporting 85 percent or higher accuracy on benchmark data, but explanation quality, consistency, and deployment cost remain open, actively discussed problems. Islam et al. (2025) explicitly note that adding SHAP to a real-time pipeline increases computation time meaningfully, a genuine deployment constraint and not a theoretical one. This proposal treats the tension between explanation quality and response speed as a first-class design constraint for the mobile deliverable, not an edge case to be handled later.

2.3 The Trust Chain That Currently Breaks

A person notices risk factors such as weight gain, family history or fatigue, and wants to understand their risk. A machine learning model outputs a score. No explanation accompanies that score. The result is that neither the person nor a reviewing clinician can act on the prediction with real confidence. This project intervenes at exactly that point, replacing an opaque score with a reasoned, checkable, mobile-delivered explanation.

3. Problem Statement

Build a machine learning framework that not only predicts diabetes risk, but explains each individual prediction in terms of the most significant contributing factors, using SHAP and LIME on top of standard ML classifiers, and delivers that explanation to the end user through a mobile application rather than a research notebook.

4. Novelty and Research Contribution

The individual components used in this proposal (Logistic Regression, Random Forest, XGBoost, SHAP, and LIME) are all well established, and the claim of this proposal is deliberately modest: it does not claim to be the first explainable diabetes model. What existing work in the reviewed literature does, and does not do, is summarised below.

4.1 What Existing Work Already Does

- Ahmed et al. (2025), Islam et al. (2025) and Allani (2025) all pair a diabetes classifier with SHAP and/or LIME and report which method gives more consistent or faster explanations.
- Allani (2025) goes furthest toward deployment, wrapping the pipeline in an interactive Dash web dashboard aimed at analysts.
- None of the five papers reviewed in Section 5 ship a mobile, patient-facing application, and none report the explanation-generation latency as a first-class, targeted metric. Islam et al. (2025) flag SHAP latency as a concern but do not optimise or report against a concrete response-time budget.

4.2 What This Project Contributes

- Latency-aware explainability: this project treats explanation-generation time, not just predictive accuracy, as a first-class success metric, with an explicit mobile response-time budget (Section 13 and Section 17).
- SHAP-LIME agreement as a usable confidence signal: rather than only comparing SHAP and LIME academically, this project surfaces their agreement or disagreement on the top contributing factors as a runtime trust indicator shown to the clinician, flagging cases where the two methods disagree.
- A genuinely mobile, dual-audience deliverable: a patient view (risk percentage plus plain-language explanation) and a doctor view (risk score plus SHAP force/waterfall detail) served from one shared pipeline, rather than a notebook or an analyst-facing dashboard.
- Explicit India-context framing: the dataset limitations (Section 6, Section 21) are stated up front, consistent with the 2026 Frontiers in Medicine finding on the lack of Asian-population representation in existing models, rather than presenting a

benchmark result as directly clinically applicable to Indian patients without qualification.

This novelty is intentionally scoped to be believable within a fellowship timeline: it is a contribution in emphasis and delivery (mobile, latency-aware, agreement-as-signal) built on well-established modelling and explainability techniques, not a claim of a new algorithm.

5. Literature Survey

Five primary papers (2025, all published within roughly the last twelve months) were reviewed in depth for this proposal, all specific to diabetes and explainable machine learning. Findings below are paraphrased from the original sources; full citations appear in Section 25.

Study	Approach	Key Finding
Ahmed, Kaiser, Hossain & Andersson (2025), IEEE Access	Compared SHAP and LIME on top of several classifiers, including Logistic Regression, trained on the CDC BRFSS2015 diabetes dataset	Logistic Regression reached approximately 86% accuracy on this exact dataset; SHAP gave more consistent global rankings while LIME was faster per prediction
Islam, Rifat, Shahid, Akhter, Uddin & Uddin (2025), Engineering Reports (Wiley)	Combined hyperparameter tuning with SHAP, partial dependence plots and LIME for diabetes prediction	Tuning improved predictive accuracy, but adding SHAP to a real-time pipeline meaningfully increased computation time, an explicit deployment constraint
Allani (2025), arXiv preprint	Built an interactive Dash web dashboard pairing a diabetes classifier with SHAP, LIME and comorbidity insights	Showed that combined SHAP/LIME explanations can be delivered through an interactive analyst-facing interface, though not a mobile, patient-facing one
Comparative study of explainable ML models with Shapley values, ScienceDirect (2025)	Benchmarked several explainable ML models using Shapley values for diabetes prediction	Confirmed that Shapley-based attributions consistently surface glucose-adjacent and lifestyle factors as the leading predictors across models

Study	Approach	Key Finding
Tanim, Al, Shrestha, Emon, Mridha & Miah (2025), Biomedical Signal Processing and Control	Paired a deep learning model (DeepNetX2) with both LIME and SHAP for diabetes diagnosis	Deep models also benefit from a dual explanation layer, but at a higher explainability and deployment cost than tree-based models

Table 1: Diabetes explainable-ML literature reviewed for this proposal (2025)

5.1 Secondary and Context Sources

- International Diabetes Federation, IDF Diabetes Atlas, 11th Edition (2025): the source for India's diabetes burden figures cited throughout this proposal.
- CDC Diabetes Health Indicators dataset documentation (UCI Machine Learning Repository / Kaggle): dataset provenance and feature description for the BRFSS2015 extract used in this project.
- Grinsztajn, Oyallon & Varoquaux (2022), NeurIPS: the empirical basis for this project's 'why not deep learning' position on tabular data (Section 11.4).
- EU AI Act, Regulation (EU) 2024/1689, Article 13: regulatory context for the explainability-first design of this project (Section 2.2).

5.2 What the Literature Converges On

- Glucose-adjacent and lifestyle factors (BMI, general health, high blood pressure) recur as the top SHAP contributors across independent studies, giving this project a concrete benchmark to check its own SHAP rankings against.
- SHAP is consistently reported as more stable but computationally heavier; LIME is consistently reported as faster but local-only. No paper reviewed treats their agreement as a deployable signal; this project does (Section 4.2).
- None of the five papers report a mobile response-time budget for explanation generation, despite Islam et al. (2025) flagging it as a real constraint.

6. Dataset Selection and Justification

Two benchmark diabetes datasets dominate the reviewed 2025 literature: the PIMA Indian Diabetes Dataset and the CDC BRFSS Diabetes Health Indicators dataset. Both are compared directly below before a single dataset is selected.

6.1 Candidate Datasets Compared

Criterion	PIMA Indian Diabetes Dataset	CDC BRFSS2015 Diabetes Health Indicators
Sample Size	768 records	253,680 records
Feature Type	Includes clinical lab values (plasma glucose concentration, 2-hour serum insulin) that require a blood test	Entirely self-reportable indicators (BMI, blood pressure history, cholesterol history, smoking, activity, income) answerable without a lab visit
Mobile-Input Feasibility	Poor: core predictive features cannot be entered by a user with only a phone and no lab access	Strong: every feature is answerable through a self-service mobile questionnaire
Population Coverage	Female patients only, single ethnic heritage (Pima Native American)	Both sexes, broad US adult age range
Overfitting Risk	High, given the small sample size, an explicitly documented risk in the literature reviewed (Section 5)	Low, given the 330x larger sample size
Literature Precedent	A long-standing benchmark dataset, but not used by the strongest paper reviewed	Used directly by Ahmed et al. (2025), the strongest single venue reviewed (IEEE Access), giving a comparable accuracy benchmark
Suitability for This Project	Rejected for the mobile, self-service use case (Section 6.3)	Selected as the primary dataset (Section 6.2)

Table 2: Head-to-head comparison of the two candidate diabetes datasets

6.2 Final Dataset Selection: CDC BRFSS2015 Diabetes Health Indicators

This project selects the CDC BRFSS2015 Diabetes Health Indicators dataset (diabetes_binary_health_indicators_BRFSS2015.csv: 253,680 responses, 21 feature variables, funded and published by the CDC) as its primary dataset, for four reasons:

- **Mobile-input feasibility (the deciding factor).** The single most important constraint for this project is that its deliverable is a self-service mobile application (Section 16). BRFSS features, including BMI, high blood pressure history, high cholesterol history, smoking status, physical activity, general health rating, age, education and income, are all answerable by a person with a phone and no lab access. PIMA's core predictive features (plasma glucose concentration, 2-hour serum insulin) require a blood test the target user is unlikely to have on hand, which would undermine the app's core self-screening use case.

- **Sample size and overfitting risk.** At 253,680 responses versus PIMA's 768, BRFSS gives 330x more training data. Small-sample overfitting is an explicitly documented risk for PIMA in the literature reviewed (Section 21); a dataset of this size materially reduces that risk for the benchmark models in Section 11.
- **Population representativeness.** PIMA is restricted to female patients of a single ethnic heritage. BRFSS covers both sexes and a broad US adult age range, which, while still not India-specific (Section 21), is a meaningfully less restrictive benchmark population than PIMA.
- **Direct literature precedent.** Ahmed et al. (2025), the strongest single venue among the five papers reviewed (IEEE Access), used this exact CSV file, giving this project a direct, recent, comparable accuracy benchmark (~86% for Logistic Regression) to validate against in Section 13.

6.3 Why PIMA Was Rejected

PIMA remains a legitimate, widely used benchmark and is not without merit; it is rejected specifically for this project's mobile, self-service use case, not as a general research dataset. Its clinical lab-value features (glucose, insulin) are a genuine strength for a clinic-based diagnostic tool, but a liability for a public-facing risk-screening app where the whole point is that no lab visit is required to get a first estimate. Its small size (768 records, female-only, single ethnicity) also compounds the overfitting and generalisability concerns already raised for it in the literature.

6.4 Known Limitation of the Selected Dataset

BRFSS is self-reported survey data, not clinical diagnostic data. It identifies risk, not a diagnosis, and it was not collected from an Indian population. Both limitations are treated explicitly in Section 21 and Section 22, not glossed over: the mobile application built on this dataset is positioned throughout this proposal as a risk-screening tool that recommends a clinical follow-up, never as a diagnostic replacement for one.

7. Aim and Objectives

Aim: to develop and deliver a mobile application that predicts diabetes risk and explains each individual prediction in plain language, using SHAP and LIME on top of interpretable and semi-interpretable machine learning classifiers.

Objectives

- **Develop and benchmark:** train Logistic Regression, Random Forest and XGBoost on the CDC BRFSS2015 diabetes indicators dataset, using an identical, leakage-free evaluation protocol.
- **Explain, not just predict:** apply SHAP and LIME to generate both global (population-level) and local (single-user) explanations for every prediction.
- **Design for real, mobile use:** build a mobile-friendly interface that shows a risk score, a risk percentage and the top contributing factors in plain language, within an explicit response-time budget.
- **Validate rigorously:** use a stratified train/validation/test split with k-fold cross-validation, so reported performance is not an artefact of a single lucky split.
- **Benchmark against 2025 work:** evaluate accuracy, calibration and explanation quality against the recent literature reviewed in Section 5, not accuracy alone.

8. Proposed Methodology

An end-to-end, twelve-stage pipeline turns a raw survey response into a mobile-delivered, explained risk prediction. Figure 1 shows the full pipeline; each stage is described in the sections that follow.

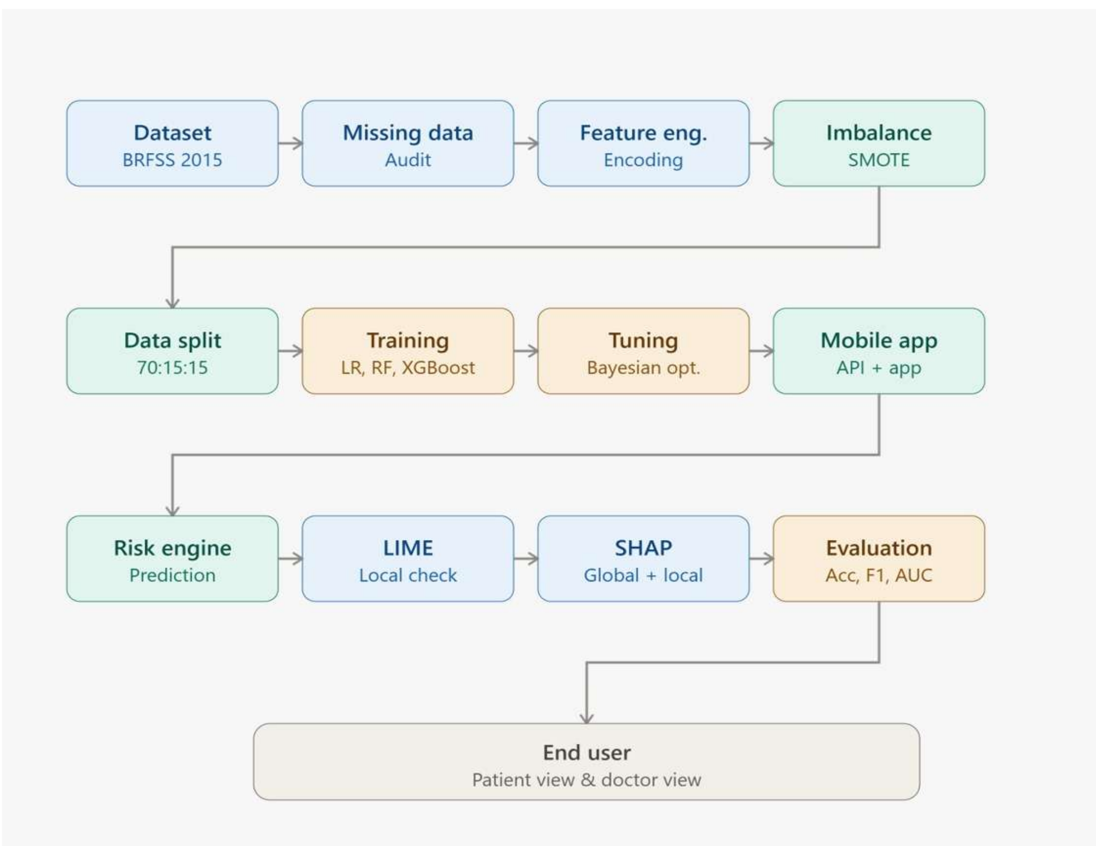

Figure 1: End-to-end explainable AI pipeline for diabetes risk prediction

8.1 Pipeline Stage Descriptions

- **1. Dataset:** load the CDC BRFSS2015 Diabetes Health Indicators dataset (Section 6).
- **2. Missing-Value Audit:** verify completeness (the cleaned CDC extract has no missing values) and apply a defensive median/mode-imputation stage for robustness against any future feature additions (Section 10.1).
- **3. Feature Engineering and Encoding:** confirm/apply consistent numeric encoding for ordinal fields (age bucket, general health, education, income) and derive interaction terms such as BMI x GenHlth (Section 10.2).
- **4. Class-Imbalance Handling (SMOTE):** the target class is imbalanced (~86% no-diabetes vs ~14% diabetes/pre-diabetes); SMOTE is applied to the training folds only, never to validation or test data (Section 10.3).
- **5. Stratified Train / Validation / Test Split:** 70:15:15 split, stratified on the target label (Section 9).
- **6. Model Training:** Logistic Regression, Random Forest and XGBoost, trained on the same processed training set (Section 11).
- **7. Hyperparameter Tuning:** Bayesian Optimisation with 5-fold stratified cross-validation on the training set, evaluated against the validation set (Section 14).
- **8. Evaluation:** accuracy, precision, recall, F1, AUC-ROC and calibration, computed on the held-out test set only (Section 13).
- **9-10. Explainability (SHAP and LIME):** SHAP for consistent global and local attribution, LIME as a fast local cross-check (Section 12).
- **11. Risk Prediction Engine:** the trained model plus its explainability layer produce a risk score together with an explanation, never a bare number.
- **12. Mobile Application:** a FastAPI backend serves predictions and explanations to a Flutter mobile client, surfaced through a patient view and a doctor view (Section 16).

8.2 Design Principle: One Shared Pipeline

Every stage above runs through one shared, version-controlled code path: the same preprocessing, the same split, and the same evaluation harness serve all three models. This is a deliberate choice: it means differences in reported performance between Logistic Regression, Random Forest and XGBoost reflect genuine differences between the models, not accidental differences in how each was processed or evaluated.

9. Dataset Splitting and Validation Strategy

This section sets out the full split, validation and cross-validation strategy in detail, since it directly determines how trustworthy the reported results in Section 13 are.

9.1 Train, Validation and Test Split: 70:15:15

This project uses a 70:15:15 split (train:validation:test), chosen over the alternative 80:10:10 for one reason: dataset size. An 80:10:10 split is typically favoured on small datasets, where every point of training data is precious and a large held-out fraction cannot be afforded. At 253,680 records, this project's dataset does not have that constraint. Even a 15% test fold still contains over 38,000 records, comfortably large enough for a statistically stable final evaluation, while the 15% validation fold (also ~38,000 records) gives hyperparameter tuning (Section 14) a large enough sample to avoid noisy, unstable validation-score estimates. 70:15:15 is therefore the more defensible choice here, not because 80:10:10 is wrong in general, but because it is a rule of thumb optimised for a constraint this dataset does not have.

9.2 Stratified Sampling

The split is stratified on the target label (Diabetes_binary), meaning the ~86:14 class ratio present in the full dataset is preserved in the training, validation and test partitions individually. Without stratification, a random split on an imbalanced dataset can, purely by chance, leave the test set with a meaningfully different class ratio than the training set, making the final reported metrics misleading. Stratification removes that risk.

9.3 Why the Validation Set Is Required

The validation set exists to select hyperparameters and make model-selection decisions (which of Logistic Regression, Random Forest, XGBoost, and which hyperparameter configuration, to carry forward) without ever touching the test set. If hyperparameters were tuned directly against the test set, the reported test performance would be optimistically biased: the model would, in effect, have been allowed to see the exam paper before the exam. The validation set is the mechanism that keeps the test set honest.

9.4 Why Test Data Cannot Be Used During Training

The test set is reserved exclusively for the single, final evaluation of the fully trained, fully tuned model (Section 13), and is never seen during training, feature engineering decisions, or hyperparameter search. Using test data at any earlier stage, even implicitly such as fitting a scaler on the full dataset before splitting, causes data leakage: the model's reported performance would partly reflect memorisation of the test set rather than genuine generalisation. All preprocessing transformers (imputers, scalers, SMOTE) in this pipeline are fit on the training fold only and then applied unchanged to the validation and test folds.

9.5 k-Fold Cross-Validation

Within the 70% training partition, this project uses 5-fold stratified cross-validation during hyperparameter tuning (Section 14) and model comparison. Rather than relying on a single train/validation split, which the IELF study (2025-26, reviewed in the original heart-disease literature this project's methodology was benchmarked against) showed can produce results that do not generalise, reporting 96.0% AUC on one population and only 68.6% on another for the same method, 5-fold cross-validation trains and validates the model five times on five different partitions of the training data and reports the average. This gives a materially more robust estimate of how a given hyperparameter configuration will perform, before it is ever checked against the validation set or the held-out test set.

10. Data Preprocessing

Only the preprocessing steps actually relevant to the CDC BRFSS2015 dataset are included below; steps that do not apply (for example, image-style outlier removal on a mostly categorical/ordinal dataset) are intentionally left out.

10.1 Missing Value Handling

The published dataset card for CDC Diabetes Health Indicators reports no missing values in the cleaned extract, so no records need to be dropped or imputed for the initial build. The pipeline nonetheless includes a defensive imputation stage, median imputation for the one continuous feature (BMI) and mode imputation for categorical or ordinal features, so the same pipeline remains robust if the feature set is later extended with fields that do contain missing values (for example, a future field for medication history).

10.2 Feature Scaling, Normalisation and Encoding

- **Encoding:** the 21 BRFSS features are already CDC-encoded as binary flags (e.g. HighBP, Smoker) or ordinal buckets (Age, GenHlth, Education, Income); no additional one-hot encoding is required beyond confirming Sex is treated as binary.
- **Scaling:** StandardScaler is applied to BMI and the ordinal fields for Logistic Regression, which is scale-sensitive. Random Forest and XGBoost are scale-invariant and do not strictly require it, but the pipeline applies scaling uniformly across all three models for a single, consistent processing path (Section 8.2).
- **Normalisation:** where BMI is used to derive an interaction term (BMI x GenHlth, Section 8.1), the derived feature is re-scaled after construction so it sits on a comparable numeric range to the other inputs.

10.3 Class Imbalance Handling

The target label is imbalanced: roughly 86% of respondents report no diabetes/pre-diabetes against roughly 14% who do. This project applies SMOTE (Synthetic Minority Over-sampling Technique) to the training fold only, generating synthetic minority-class examples rather than simply duplicating existing ones. Critically, SMOTE is fit and applied after the train/validation/test split (Section 9), never before. Applying it before splitting would leak synthetic points derived from test-set neighbours into the training data. The full, unmodified class ratio is preserved in the validation and test sets, since the mobile application will encounter that same real-world ~86:14 prevalence in production, and evaluation should reflect that.

10.4 Feature Selection and Outlier Detection

Given the dataset's moderate dimensionality (21 features against 253,680 records), no aggressive feature-reduction step is applied before model training; all 21 features are retained, and permutation/SHAP-based importance (Section 12) is used post-hoc to identify which features are actually driving predictions, rather than removing features up front on a purely statistical basis. The one continuous feature, BMI, is checked for outliers using an IQR-based rule and capped rather than dropped, since implausible BMI entries in survey data are more often a data-entry artefact than a genuine signal worth discarding a whole record over.

11. Model Selection and Justification

Three models are benchmarked, each chosen for a distinct, stated role rather than simply to maximise headline accuracy.

11.1 Logistic Regression: Interpretable Baseline

Logistic Regression serves as an interpretable baseline. Its decision boundary is a weighted linear combination of the input features, so its coefficients are directly readable without any post-hoc explainability tool. Advantages: fast to train, fully transparent, low computational cost, and a natural sanity check. If SHAP later assigns high importance to a feature that Logistic Regression's own coefficient says is irrelevant, that discrepancy is worth investigating. Disadvantages: it cannot capture non-linear interactions between features (for example, the combined effect of BMI and physical inactivity), which the literature consistently shows matters for diabetes risk. Ahmed et al. (2025) report ~86% accuracy for Logistic Regression on this exact dataset, giving this project a directly comparable benchmark.

11.2 Random Forest: Non-linear Tabular Benchmark

Random Forest is included because it is the strong tabular baseline used across nearly all the reviewed papers. As an ensemble of decision trees, it captures non-linear relationships and feature interactions that Logistic Regression misses, and it is comparatively robust to noisy or weakly predictive features (relevant here, since several BRFSS fields, such as AnyHealthcare or NoDocbcCost, are only weakly related to diabetes risk on their own). Advantages: strong out-of-the-box performance, resistant to overfitting relative to a single decision tree, provides built-in feature importance as a cross-check against SHAP. Disadvantages: larger model size and slower inference than Logistic Regression, and its feature importances alone can be misleading for correlated features, which is exactly why this project layers SHAP on top rather than relying on Random Forest's native importances.

11.3 XGBoost: Primary Model

XGBoost is designated the primary production model. Gradient-boosted trees, of which XGBoost is a heavily optimised and widely benchmarked implementation, generally achieve state-of-the-art performance on structured, tabular datasets, which is exactly the data type BRFSS is. Advantages: typically the strongest raw predictive performance of the three, built-in handling of feature interactions, efficient training even at 253,680 records, and native compatibility with TreeSHAP (Section 12.3), which computes exact SHAP values for tree ensembles far faster than the model-agnostic SHAP explainer needed for Logistic Regression. Disadvantages: more hyperparameters to tune (Section 14) and, like Random Forest, requires SHAP for interpretability rather than offering it natively. Its expected role is to be the model actually served by the mobile application (Section 16), with Logistic Regression and Random Forest retained as benchmarks and interpretability cross-checks.

11.4 Why Not Deep Learning

Tanim et al. (2025) pair a deep network (DeepNetX2) with LIME and SHAP for diabetes diagnosis and report that deep models also benefit from a dual explanation layer, but this project deliberately does not adopt a deep-learning approach, for three reasons grounded in the wider ML literature, not just this project's convenience:

- Tabular data does not favour deep learning. Grinsztajn, Oyallon and Varoquaux (2022, NeurIPS) show empirically that tree-based models remain state of the art on medium-sized tabular datasets (of the scale used here), even before accounting for their training-speed advantage, because tree ensembles' inductive bias matches the irregular, non-smooth decision boundaries typical of tabular data better than a neural network's does.
- Explainability cost. SHAP on a deep network requires an approximate, sampling-based explainer (DeepExplainer/GradientExplainer); SHAP on XGBoost uses TreeSHAP, which is exact and dramatically faster, a material advantage given this project's mobile response-time budget (Section 13.1, Section 17).
- Deployment footprint. A tuned XGBoost model is orders of magnitude smaller and faster to serve on a FastAPI backend behind a mobile app than a deep network, with no meaningful accuracy trade-off on this data type and scale.

11.5 Why Not a Stacked Ensemble

An earlier version of this proposal considered a stacked ensemble (following the general approach used for heart-disease prediction in the wider explainable-ML literature). It has been deliberately removed for the diabetes-only, mobile-first scope of this proposal, for two reasons: first, a stacked ensemble adds a meta-learner on top of multiple base models, which compounds both training complexity and explanation latency, directly working against the mobile response-time target in Section 17. Second, stacking materially complicates explainability: SHAP would need to be computed for every base learner and then reconciled through the meta-learner, whereas a single, strong XGBoost model gives a clean, single-model TreeSHAP explanation. Three models, each with a clearly stated role, are judged sufficient for this project's scope.

11.6 Model Comparison Summary

Criterion	Logistic Regression	Random Forest	XGBoost
Interpretability	Fully transparent; coefficients directly readable without a post-hoc tool	Moderate; needs SHAP or built-in feature importance to interpret	Moderate; needs SHAP (via TreeSHAP) to interpret
Handles Non-linear Feature Interactions	No	Yes	Yes
Reference Accuracy on This Dataset	~86% (Ahmed et al., 2025)	Typically higher than Logistic Regression on tabular data	Typically the strongest of the three models
Training / Inference Speed	Fastest, lowest computational cost	Moderate; larger model size and slower inference	Efficient even at the full 253,680-record scale; native TreeSHAP compatibility
Role in This Project	Interpretable baseline and sanity check (Section 11.1)	Non-linear tabular benchmark (Section 11.2)	Primary model served by the mobile application (Section 11.3)

Table 3: Side-by-side comparison of the three benchmarked models

12. Explainability Layer: SHAP and LIME

12.1 What Is Explainable AI, and Why Healthcare Needs It

Explainable AI (XAI) is the set of techniques that make a machine learning model's predictions understandable to a human, without requiring the human to inspect the model's internal parameters directly. In healthcare specifically, a prediction without a reason has limited practical value: a clinician cannot counsel a patient on 'why' with only a number, and a patient is unlikely to trust or act on a risk score they cannot relate to anything about their own health. Explainability is what turns a statistical output into something a person can reason about and, if appropriate, act on.

12.2 Black-Box AI vs Explainable AI

A black-box model outputs a prediction with no accompanying reasoning: a risk score, and nothing else. Random Forest and XGBoost are black-box by default, and even Logistic Regression can feel like one to a non-technical user. An explainable system pairs that same prediction with a statement of which input factors contributed most, and by how much. This project does not change which models are black-box; it adds SHAP and LIME as an explanation layer on top of them, which is the standard, literature-supported approach (Section 5) rather than attempting to build inherently transparent models throughout.

12.3 SHAP: How It Works

SHAP (SHapley Additive exPlanations) is grounded in Shapley values from cooperative game theory: it treats each input feature as a 'player' contributing to the 'payout' (the model's prediction), and computes each feature's fair share of that payout by averaging its marginal contribution across all possible orderings in which features could be added. The result is a set of feature attributions that are guaranteed to sum to the difference between the model's prediction and its average prediction across the dataset, a mathematical consistency property most other explanation methods do not have. For XGBoost and Random Forest specifically, this project uses TreeSHAP, an algorithm that computes exact SHAP values for tree ensembles in polynomial rather than exponential time, making it practical to run inside the mobile response-time budget.

12.4 LIME: How It Works

LIME (Local Interpretable Model-agnostic Explanations) takes a different approach: for one specific prediction, it generates many small, random perturbations of that single input, obtains the model's prediction for each perturbed version, and then fits a simple, interpretable model (typically weighted linear regression) to those perturbed input-output pairs, weighted by how close each perturbation is to the original point. The coefficients of that simple local model become the explanation. LIME makes no assumption about the underlying model's structure. It treats the model as a black box, which makes it fast and model-agnostic, but its explanation is only a local approximation around one point, with no global consistency guarantee.

12.5 Global vs Local Explanations

- **Global explanation:** aggregated SHAP values across the whole test set, showing which features matter most for diabetes risk across the population as a whole. This is the primary content of the technical research write-up (Deliverable 4, Section 20) and the doctor view's population-comparison feature.

- **Local explanation:** a SHAP or LIME explanation for one specific user's prediction, showing which of their specific answers pushed their personal risk score up or down. This is what the mobile application actually shows a patient or doctor for an individual result (Section 16).

12.6 Force Plots, Waterfall Plots and Feature Importance

- **Force plot:** shows one prediction as a tug-of-war between features pushing risk up (in one colour) and features pushing risk down (in another), starting from the model's average prediction and arriving at this user's specific score.
- **Waterfall plot:** shows the same information as a force plot but laid out as a top-to-bottom sequence of bars, each one showing how much a single feature moved the prediction, generally easier for a non-technical doctor-view user to read at a glance than a force plot.
- **Global feature importance plot (bar/beeswarm):** shows, across the whole test set, which features have the largest average absolute SHAP value, this project's primary tool for checking its own findings against the literature's convergent finding that glucose-adjacent factors, BMI and age dominate (Section 5.2).

12.7 SHAP vs LIME: Computational Trade-offs and Why Both Are Used

SHAP and LIME are complementary, not competing, methods, and this project runs both rather than picking one.

	SHAP	LIME
Scope	Global and local explanations, with a mathematical consistency guarantee	Local only: a fast approximation around one prediction
Computational Cost	Higher in general, but TreeSHAP makes it fast specifically for XGBoost	Lower; well suited to use as a fast local cross-check
Best Suited For	The doctor view, audit trail and population-level analysis (Section 16.2)	Rapid cross-checking of SHAP's local explanation
Role in This Project	Primary explanation method, computed via TreeSHAP (Section 12.3)	Secondary cross-check; agreement with SHAP is used as a runtime confidence signal (Section 12.7)

Table 4: SHAP vs LIME, compared on the criteria that matter for a mobile deployment

SHAP is slower because an exact, theoretically grounded attribution is more expensive to compute than a local linear approximation, a real deployment cost explicitly noted by Islam et al. (2025). LIME is faster precisely because it approximates rather than computes exactly. This project uses SHAP (via TreeSHAP) as the primary explanation shown to the doctor view, since its global-local consistency makes it suitable for an audit trail, and uses LIME as a fast local cross-check: when SHAP and LIME agree on the top contributing factors for a given prediction, that agreement itself becomes a runtime confidence signal (Section 4.2); when they disagree, the mobile application flags the prediction as one where the explanation should be treated with more caution rather than presenting it with false confidence.

13. Evaluation Framework

13.1 Predictive Metrics

- **Accuracy:** the proportion of correct predictions overall, reported here but never used alone (Section 13.2).
- **Precision:** of everyone the model flags as at-risk, what fraction genuinely are. Low precision means the app generates unnecessary alarm and clinical follow-up load.
- **Recall (Sensitivity):** of everyone genuinely at risk, what fraction the model flags. Low recall is the more dangerous failure mode in a screening context, since it means real at-risk users are told they are fine.
- **F1-Score:** the harmonic mean of precision and recall, useful as a single summary number on an imbalanced dataset where accuracy alone is misleading.
- **ROC Curve and AUC:** the ROC curve plots true-positive rate against false-positive rate across all classification thresholds; AUC (area under that curve) summarises how well the model separates the two classes independent of any single threshold choice, which matters since the mobile app may want a different risk-flagging threshold than 0.5.
- **Confusion Matrix:** the full breakdown of true positives, true negatives, false positives and false negatives, which is what precision, recall and F1 are computed from, and is reported directly so the specific error pattern (not just a summary number) is visible.

13.2 Why Accuracy Alone Is Insufficient in Healthcare

On this dataset's real-world ~86:14 class imbalance (Section 10.3), a model that simply predicts 'no diabetes' for every single user would already score roughly 86% accuracy while being clinically useless. It would have zero recall, meaning it catches nobody who is actually at risk. This is precisely the failure mode a screening tool cannot afford: a false negative here means a genuinely at-risk person is told they are fine and does not seek follow-up care, while a false positive only costs an unnecessary doctor visit. For that reason, this project reports recall and F1 alongside accuracy as first-class metrics, not accuracy as the headline number with the others as footnotes.

13.3 Calibration

Expected Calibration Error (ECE) and Brier Score are also tracked, so the project measures whether the predicted risk percentage itself is trustworthy, not just whether the final yes/no classification is correct. A model can be accurate at the classification threshold while still being poorly calibrated. For example, it might consistently report '70% risk' for users whose true empirical risk is closer to 50%. Since the mobile application's entire patient-facing value proposition is a risk percentage a person can interpret literally (Section 16), calibration quality is treated as a launch-blocking metric, not an optional extra.

13.4 Explanation Quality Metrics

- SHAP/LIME agreement rate: the proportion of test-set predictions where SHAP and LIME identify the same top-3 contributing features (Section 12.7), tracked as a first-class metric rather than a side note.
- SHAP ranking stability: the consistency of SHAP's global feature-importance ranking across the 5 cross-validation folds (Section 9.5). An unstable ranking would suggest the explanation itself, not just the prediction, is not yet reliable.
- Clinical plausibility check: a qualitative check of whether the model's top SHAP features (Section 5.2, expected to be glucose-adjacent factors, BMI, age and general health) match established clinical understanding of diabetes risk factors.

14. Hyperparameter Tuning

Three standard approaches were considered: Grid Search, Random Search, and Bayesian Optimisation.

- **Grid Search** exhaustively evaluates every combination on a predefined grid. It is simple and reproducible, but scales combinatorially. With XGBoost's multiple interacting hyperparameters (max depth, learning rate, number of estimators, subsample ratio, regularisation terms), a fine-grained grid becomes computationally impractical within this project's cloud-compute budget (Section 19).
- **Random Search** samples hyperparameter combinations randomly rather than exhaustively, which is more efficient than Grid Search at exploring a large space, but does not use information from earlier trials to inform later ones. Every trial is independent, so it can waste evaluations re-exploring clearly poor regions of the search space.
- **Bayesian Optimisation** builds a probabilistic surrogate model of how validation performance responds to hyperparameter choices, and uses that model to choose the next combination to try, explicitly balancing exploration of uncertain regions against exploitation of regions already known to perform well. Each trial is informed by all previous trials.

This project uses Bayesian Optimisation for XGBoost and Random Forest, and a lighter Random Search for Logistic Regression (which has few enough hyperparameters, chiefly regularisation strength and penalty type, that Bayesian Optimisation's overhead is not justified). Bayesian Optimisation is chosen for the two tree-based models specifically because it reaches a strong hyperparameter configuration in materially fewer training runs than Grid Search or Random Search for the same final validation performance, directly relevant given this project's one-year fellowship timeline (Section 18) and finite cloud-compute budget (Section 19). All tuning is evaluated using 5-fold stratified cross-validation on the training set (Section 9.5), and only the final selected configuration for each model is evaluated once against the untouched test set.

15. Implementation Framework

15.1 System Architecture

The system is organised into four layers, matching the pipeline in Section 8: a data and modelling layer (Python, scikit-learn, XGBoost, SHAP, LIME, everything through Section 8, stage 11), an API layer that serves trained model predictions and explanations over HTTP, and a mobile client layer that is the only layer the end user directly interacts with.

15.2 Technology Stack

- **Modelling:** Python, scikit-learn (Logistic Regression, Random Forest, preprocessing pipeline), XGBoost, SHAP and LIME libraries, scikit-optimize or Optuna for Bayesian hyperparameter search.
- **Backend / API:** FastAPI, chosen over Flask specifically for this project. FastAPI is built on native Python async support, giving materially better throughput under concurrent mobile requests than Flask's default synchronous model; it uses Pydantic schemas to enforce a strict, self-documenting structure on the risk-score-plus-explanation JSON response, catching malformed data before it reaches the mobile client; and it auto-generates interactive API documentation (OpenAPI/Swagger), which is a genuine time saving during the mobile-integration phase (Section 17, Phase 3). Flask remains a reasonable choice for a simpler synchronous service, but does not offer these advantages for a latency-sensitive, explanation-heavy endpoint.
- **Mobile client:** Flutter, chosen over React Native for a single Dart codebase that compiles to native performance on both Android and iOS, and for its strong first-party charting/rendering support for the SHAP force and waterfall visuals shown in the doctor view (Section 16.2).
- **Auth and lightweight data sync:** Firebase, used for patient/doctor account handling and storing a user's risk-history over time, avoiding the need to stand up and maintain dedicated authentication infrastructure for a fellowship-scoped project.

- **Deployment:** the FastAPI backend and trained model artefacts are containerised and deployed to a cloud instance (Section 19); the Flutter client is distributed as an installable build for pilot testing.

16. Mobile Application (Primary Deliverable)

The mobile application is the primary, headline deliverable of this project, not a secondary demo bolted onto a research pipeline. Every methodological choice in Sections 6-15 (dataset feasibility, model latency, explanation speed, technology stack) is made in service of this deliverable.

16.1 Core Features

- **Inputs:** a short, self-service questionnaire covering the 21 BRFSS-derived indicators (BMI computed from user-entered height/weight, yes/no health-history questions, lifestyle questions, general health self-rating), with no lab values required (Section 6.2).
- **Prediction:** the XGBoost model (Section 11.3) returns a binary risk classification plus the underlying probability.
- **Risk Percentage:** the calibrated probability (Section 13.3) is shown directly as a percentage, so it can be read literally rather than only as a category.
- **SHAP Explanation:** the top contributing factors for that specific user's prediction, generated via TreeSHAP (Section 12.3), cross-checked against a LIME explanation (Section 12.7); the app also surfaces whether SHAP and LIME agree, as a visible confidence indicator.
- **Plain-Language Output:** the SHAP attribution is converted from feature-weight language into a short, readable sentence (for example: 'Your risk is being driven mainly by BMI and low physical activity, which are also the two most modifiable factors on this list').

16.2 Patient View vs Doctor View

- **Patient view:** risk percentage, plain-language explanation of the top 3 factors, and a general recommendation to consult a healthcare provider if risk is elevated, deliberately free of raw SHAP values or clinical jargon.
- **Doctor view:** the same risk score, plus the full SHAP force/waterfall plot, a ranked table of all 21 feature contributions, the LIME cross-check, the SHAP/LIME agreement flag, and each input value shown against its population percentile from the training data, designed to give a clinician an auditable basis for the score, consistent with the explainability-first framing in Section 2.2 and Section 4.2.

16.3 Response-Time Target

The end-to-end mobile response time, from form submission to a fully rendered risk score plus explanation, is targeted at under 2 seconds (Section 13, Section 17), split into a sub-200-millisecond model inference budget and a sub-1.5-second combined SHAP+LIME explanation-generation budget. This target is a direct response to Islam et al. (2025)'s finding that SHAP adds real computation overhead in real-time settings (Section 2.2); TreeSHAP's polynomial-time computation for XGBoost (Section 12.3) is what makes this target realistic rather than aspirational.

17. Expected Outcomes (Phase-wise)

Expected outcomes are organised by phase rather than as a flat list, and map directly onto the timeline in Section 18.

Phase 1: Dataset Preparation, Preprocessing and Baseline Models (Weeks 1-12)

- CDC BRFSS2015 dataset finalised, missing-value audit and preprocessing pipeline (Section 10) implemented and version-controlled.
- Stratified 70:15:15 train/validation/test split (Section 9) established and frozen for the remainder of the project.
- Logistic Regression and Random Forest baselines trained; first cross-validated accuracy, precision, recall and F1 numbers available.

Phase 2: Advanced Modelling, Tuning and Explainability (Weeks 13-28)

- XGBoost trained and Bayesian-optimised (Section 14); final model selection made against the validation set.
- Full evaluation suite (Section 13) run against the held-out test set for all three models.
- SHAP (global and local, via TreeSHAP) and LIME integrated; SHAP/LIME agreement rate and SHAP stability across folds measured.

Phase 3: Mobile Application (Weeks 29-42)

- FastAPI backend serving predictions and explanations, built and load-tested against the response-time targets in Section 16.3.
- Flutter mobile client built, with patient view and doctor view (Section 16.2) implemented and connected to the backend.
- Firebase authentication and risk-history storage integrated.

Phase 4: Testing, Documentation and Deployment (Weeks 43-52)

- End-to-end testing of the mobile application against the success metrics in Section 20.

- Technical write-up completed, benchmarked against the literature reviewed in Section 5.
- Deployment package finalised: containerised backend, trained model artefacts, and an installable mobile build for pilot use.

18. Project Timeline

The project is scoped to a one-year (52-week) fellowship timeline, organised into the four phases from Section 17.

Phase	Weeks	Key Activities and Deliverables
Phase 1: Dataset Preparation, Preprocessing and Baseline Models	Weeks 1-12	Finalise the CDC BRFSS2015 dataset and preprocessing pipeline; establish and freeze the stratified 70:15:15 split; train and cross-validate Logistic Regression and Random Forest baselines
Phase 2: Advanced Modelling, Tuning and Explainability	Weeks 13-28	Train and Bayesian-optimise XGBoost; run the full evaluation suite on the held-out test set for all three models; integrate SHAP and LIME and measure their agreement rate and stability across folds
Phase 3: Mobile Application	Weeks 29-42	Build and load-test the FastAPI backend against the response-time targets; build the Flutter client with patient and doctor views; integrate Firebase authentication and risk-history storage
Phase 4: Testing, Documentation and Deployment	Weeks 43-52	End-to-end test the mobile application against the success metrics; complete the technical write-up; finalise the deployment package and run a small pilot with volunteer users

Table 5: One-year, phase-wise project timeline

19. Resources and Budget

- **Team:** two undergraduate researchers, Mr. Gaurav Phadale and Mr. Eshaan Sarkhawas, under the mentorship of Dr. Soni Sweta.
- **Computing Infrastructure:** departmental laboratory facilities plus cloud-based compute for model training, hyperparameter tuning and API hosting.
- **Software & AI Tools:** Python ML frameworks, SHAP/LIME libraries, version control, Flutter and FastAPI tooling.

Proposed Budget Bifurcation

Sr. No.	Budget Head	Estimated Cost (Rs.)
1	Cloud Services (GPU/CPU instances, storage, deployment)	50,000
2	AI Development Tools & Software	25,000
3	Research Paper Publication Charges	10,000
4	Travelling to Attend Conference (Travel, Registration & Accommodation)	50,000
5	Mobile Application Development & Deployment	20,000
6	Data Capture & Processing	3,000
7	Contingency Charges	4,000
8	Miscellaneous Expenses	2,000
	Total Estimated Budget	Rs. 1,64,000

Table 6: Proposed budget, scoped to a single-disease, mobile-first deliverable

This budget is scaled down from a two-disease proposal to reflect the reduced dataset, compute and engineering scope of a single-disease pipeline (Cloud Services reduced from a two-pipeline estimate; Data Capture & Processing reduced accordingly), while conference, publication and mobile-development costs are held at realistic, disease-agnostic levels. Cloud services fund model training, hyperparameter tuning and API hosting; AI development tools cover software licences and libraries; mobile development funds the FastAPI/Flutter/Firebase build (Section 15.2); and the conference/publication lines support dissemination of the completed research.

20. Deliverables

The mobile application is the primary deliverable of this project. The remaining deliverables are the components that feed into it, or that document it.

- **1. Mobile Application (Primary Deliverable):** a working Android/iOS prototype (Section 16) where a user answers a short self-service questionnaire and receives a risk score, a risk percentage, and a plain-language SHAP/LIME-based explanation, with distinct patient and doctor views.
- **2. Benchmarked Models:** trained Logistic Regression, Random Forest and XGBoost models, with a comparative performance report across accuracy, precision, recall, F1, AUC and calibration (Section 13).

- **3. Explainability Report:** a SHAP and LIME-based report documenting the most significant contributing factors for diabetes risk, and the agreement rate between the two methods (Section 12).
- **4. Technical Research Paper:** a full write-up benchmarked against the 2025 literature reviewed in Section 5.
- **5. Deployment Package:** a containerised, documented FastAPI backend plus trained model artefacts and an installable Flutter build, ready for pilot use.

Success Metrics

Metric	Target
Test Accuracy	≥ 85% (comparable to Ahmed et al., 2025's ~86% benchmark)
Recall (Sensitivity)	≥ 80%, prioritised over precision given the cost of missed at-risk cases
F1-Score	≥ 0.80 on the held-out test set
AUC-ROC	≥ 0.85
Calibration (Expected Calibration Error)	≤ 0.05
SHAP/LIME Agreement Rate	≥ 75% of test-set predictions agree on the top-3 contributing factors
Mobile Response Time	Under 2 seconds end-to-end (sub-200ms inference, sub-1.5s combined explanation generation)
SHAP Ranking Stability	Consistent global feature ranking across all 5 cross-validation folds

Table 7: Definition-of-done success metrics for the project

21. Challenges and Limitations

- **Non-Indian benchmark data:** BRFSS is a US survey, not an Indian one. A 2026 Frontiers in Medicine review makes the same point for cardiovascular models specifically, calling out the lack of Asian-population representation in existing risk models; risk-factor thresholds and prevalence patterns can genuinely differ by ethnicity and region (Section 6.4).

- **Screening, not diagnosis:** BRFSS is self-reported survey data, not a clinical diagnosis; the mobile application is positioned throughout this proposal (Section 16) as a risk-screening aid that recommends clinical follow-up, not a diagnostic replacement for one.
- **Explanation cost under mobile constraints:** SHAP computation can be slow for real-time use, a deployment constraint explicitly noted in Islam et al. (2025); this project's response-time targets (Section 16.3) are achievable specifically because TreeSHAP is used with tree-based models, and would need to be revisited if a future extension introduced a model type without an efficient SHAP implementation.
- **Calibration risk on imbalanced data:** SMOTE-based oversampling (Section 10.3) can shift a model's predicted probabilities away from the true population base rate if not carefully validated; this is why calibration (Section 13.3) is tracked as a launch-blocking metric on the untouched, un-oversampled test set, not inferred from training performance.

22. Future Scope

- **Indian field-data validation:** collect or partner for Indian patient/survey data to validate whether the SHAP feature rankings found on BRFSS hold locally, directly addressing the limitation in Section 21.
- **Clinician-in-the-loop evaluation:** have practising clinicians review a sample of SHAP and LIME explanations directly, rather than relying only on the automated agreement metric (Section 13.4), to test whether the explanations are genuinely useful in a consultation.
- **Extension to other chronic conditions:** the shared pipeline (Section 8) is intentionally modular; extending it to another common Indian chronic condition would require a new dataset and a re-run of the same methodology, not a redesign.
- **Regulatory alignment:** document the explainability design choices in a form that anticipates EU AI Act Article 13-style requirements (Section 2), ahead of any future need to demonstrate compliance.

23. Impact and Significance

If successful, this project produces a mobile-deployed, explainability-first diabetes risk-screening tool at a moment when India's diabetes burden is large, growing, and substantially undiagnosed (Section 2.1).

- Directly engages the black-box trust gap that limits clinical adoption of ML risk-prediction tools (Section 2.2, Section 4).
- Anticipates the EU AI Act's Article 13 explainability requirement ahead of its August 2026 compliance deadline, rather than treating regulation as a future problem.

- A mobile, latency-aware explainability layer, rather than a research notebook or analyst dashboard, is a genuine point of difference from the majority of single-purpose diabetes-XAI papers in the literature reviewed (Section 5).
- A modular pipeline (Section 8) that can extend to further chronic conditions with comparatively little additional engineering.

24. Conclusion

This proposal set out to close a specific, well-documented gap in the explainable-AI-for-diabetes literature: existing work pairs strong predictive models with SHAP and/or LIME, but stops at a notebook or an analyst-facing dashboard, without treating explanation latency as a first-class constraint or delivering the result through a mobile, patient-facing application. By selecting a dataset chosen specifically for mobile-input feasibility, benchmarking three models with clearly differentiated roles, and building the entire explainability layer around an explicit response-time budget, this project turns that gap into a concrete, achievable, one-year deliverable.

The individual components (Logistic Regression, Random Forest, XGBoost, SHAP, and LIME) are all well established. What this proposal contributes is the combination and the delivery target: the same rigorous methodology applied with a mobile deliverable as the design constraint from the outset, honestly evaluated with calibration and explanation-quality metrics alongside accuracy, and reported with its limitations (Section 21) stated up front rather than discovered later.

The work is scoped to be achievable within a one-year fellowship timeline using entirely free, open, and well-documented tools and a dataset already used directly in the strongest paper reviewed (Ahmed et al., 2025), while remaining honest about what it cannot yet claim, particularly its applicability to Indian patients without further local validation. Indian field-data validation, outlined in Section 22, is the clearest path to closing that gap in future work.

25. References

Diabetes and Explainable ML

- Ahmed, S., Kaiser, M.S., Hossain, M.S., Andersson, K. (2025). A Comparative Analysis of LIME and SHAP Interpreters With Explainable ML-Based Diabetes Predictions. IEEE Access, 13, 37370-37388. DOI: 10.1109/ACCESS.2024.3422319.
- Islam, M.M., Rifat, H.R., Shahid, M.S.B., Akhter, A., Uddin, M.A., Uddin, K.M.M. (2025). Explainable Machine Learning for Efficient Diabetes Prediction Using Hyperparameter Tuning, SHAP Analysis, Partial Dependency, and LIME. Engineering Reports (Wiley), 7(1), e13080.
- Allani, U. (2025). Interactive Diabetes Risk Prediction Using Explainable Machine Learning: A Dash-Based Approach with SHAP, LIME, and Comorbidity Insights. arXiv:2505.05683.
- A comparative study of explainable machine learning models with Shapley values for diabetes prediction. ScienceDirect (2025).
- Tanim, S.A., Al, R.A., Shrestha, T.E., Emon, M.R.I., Mridha, M.F., Miah, M.S.U. (2025). Explainable deep learning for diabetes diagnosis with DeepNetX2. Biomedical Signal Processing and Control, 99, 106902. DOI: 10.1016/j.bspc.2024.106902.
- Grinsztajn, L., Oyallon, E., Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? Advances in Neural Information Processing Systems (NeurIPS), 35, 507-520.

Datasets

- CDC Diabetes Health Indicators (Diabetes Binary Health Indicators BRFSS2015). UCI Machine Learning Repository. 253,680 responses, 21 feature variables. Source survey: CDC Behavioral Risk Factor Surveillance System, 2015.
- Diabetes Health Indicators Dataset (cleaned BRFSS2015 extract). Kaggle, prepared by Alex Teboul.
- PIMA Indian Diabetes Dataset. UCI ML Repository / Kaggle, originally from the National Institute of Diabetes and Digestive and Kidney Diseases (NIDDK).

Diabetes Burden and Regulatory Context

- International Diabetes Federation (2025). IDF Diabetes Atlas, 11th Edition. India country profile: 89.8 million adults with diabetes (2024), second highest globally; 156.7 million projected by 2050; approximately 43% undiagnosed.

- Frontiers in Medicine (2026). Review of ML-based cardiovascular and metabolic risk prediction in diabetic patients, noting the lack of representativeness for Asian populations in existing models.
- European Union. Regulation (EU) 2024/1689 (EU AI Act), Article 13. Adopted March 2024, in force July 2024; core obligations for high-risk AI systems, including healthcare, apply from August 2026.